

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_CMDS 10
#define MAX_ARGS 32
#define BUF_SIZE 1024

// Helper function to trim spaces
char *trim_space(char *str) {
    while (*str == ' ' || *str == '\t') str++;
    if (*str == 0) return str;
    char *end = str + strlen(str) - 1;
    while (end > str && (*end == ' ' || *end == '\t')) end--;
    end[1] = '\0';
    return str;
}

// Applies input/output/error redirections to a single command stage
void apply_redirection(char *cmd_buf) {
    char *input_file = NULL;
    char *output_file = NULL;
    int append_out = 0;
    int merge_stderr = 0;
    char *stderr_file = NULL;
    int append_err = 0;

    // 1. Check for combined shorthand redirection (&> file)
    char *amp_out_pos = strstr(cmd_buf, "&>");
    if (amp_out_pos) {
        *amp_out_pos = '\0';
        output_file = trim_space(amp_out_pos + 2);
        merge_stderr = 1;
    }
}
```

```

// 2. Check for stderr redirection to stdout (2>&1)
char *merge_pos = strstr(cmd_buf, "2>&1");
if (merge_pos) {
    *merge_pos = '\0';
    merge_stderr = 1;
}

// 3. Check for standard stderr redirection (2>> or 2>)
if (!merge_stderr) {
    char *err_app_pos = strstr(cmd_buf, "2>>");
    char *err_tr_pos = strstr(cmd_buf, "2>");
    if (err_app_pos) {
        append_err = 1;
        *err_app_pos = '\0';
        stderr_file = trim_space(err_app_pos + 3);
    } else if (err_tr_pos) {
        append_err = 0;
        *err_tr_pos = '\0';
        stderr_file = trim_space(err_tr_pos + 2);
    }
}

// 4. Check for stdout redirection (>> or >)
if (!output_file) {
    char *out_app_pos = strstr(cmd_buf, ">>");
    char *out_tr_pos = strchr(cmd_buf, '>');
    if (out_app_pos) {
        append_out = 1;
        *out_app_pos = '\0';
        output_file = trim_space(out_app_pos + 2);
    } else if (out_tr_pos) {
        append_out = 0;
        *out_tr_pos = '\0';
        output_file = trim_space(out_tr_pos + 1);
    }
}
}

```

```

// 5. Check for input redirection (<)
char *in_pos = strchr(cmd_buf, '<');
if (in_pos) {
    *in_pos = '\0';
    input_file = trim_space(in_pos + 1);
}

// Process Input Redirection
if (input_file && strlen(input_file) > 0) {
    int in_fd = open(input_file, O_RDONLY);
    if (in_fd < 0) {
        perror("Input file open error");
        exit(EXIT_FAILURE);
    }
    dup2(in_fd, STDIN_FILENO);
    close(in_fd);
}

// Process Output Redirection
if (output_file && strlen(output_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (append_out ? O_APPEND : O_TRUNC);
    int out_fd = open(output_file, flags, 0644);
    if (out_fd < 0) {
        perror("Output file open error");
        exit(EXIT_FAILURE);
    }
    dup2(out_fd, STDOUT_FILENO);
    close(out_fd);
}

```

```

// Process Stderr Redirection or Merging
if (merge_stderr) {
    // Merge stderr into stdout (2>&1)
    dup2(STDOUT_FILENO, STDERR_FILENO);
} else if (stderr_file && strlen(stderr_file) > 0) {
    int flags = O_WRONLY | O_CREAT | (append_err ? O_APPEND : O_TRUNC);
    int err_fd = open(stderr_file, flags, 0644);
    if (err_fd < 0) {
        perror("Stderr file open error");
        exit(EXIT_FAILURE);
    }
    dup2(err_fd, STDERR_FILENO);
    close(err_fd);
}

// Executes pipeline integrated with stream redirections
void execute_complex_pipeline(char *line) {
    char *cmds[MAX_CMDS];
    int num_cmds = 0;

    // Split pipeline by '|'
    char *token = strtok(line, "|");
    while (token != NULL && num_cmds < MAX_CMDS) {
        cmds[num_cmds++] = trim_space(token);
        token = strtok(NULL, "|");
    }

    if (num_cmds == 0) return;

    int num_pipes = num_cmds - 1;
    int pipefds[2 * num_pipes];

    for (int i = 0; i < num_pipes; i++) {
        if (pipe(pipefds + i * 2) < 0) {
            perror("Pipe allocation error");
            return;
        }
    }
}

```

```

    }
}

for (int i = 0; i < num_cmds; i++) {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork error");
        return;
    } else if (pid == 0) {
        // Pipeline Input/Output Wiring
        if (i != 0) {
            dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
        }
        if (i != num_cmds - 1) {
            dup2(pipefds[i * 2 + 1], STDOUT_FILENO);
        }

        // Close all pipe descriptors in child
        for (int j = 0; j < 2 * num_pipes; j++) {
            close(pipefds[j]);
        }

        // Apply specific redirections (<, >, 2>&1, &>, etc.)
        apply_redirection(cmds[i]);

        // Tokenize command arguments
        char *args[MAX_ARGS];
        int arg_count = 0;
        char *arg_tok = strtok(cmds[i], " \\t");
        while (arg_tok != NULL && arg_count < MAX_ARGS - 1) {
            args[arg_count++] = arg_tok;
            arg_tok = strtok(NULL, " \\t");
        }
        args[arg_count] = NULL;

        if (arg_count == 0) exit(EXIT_SUCCESS);
    }
}

```

```

        execvp(args[0], args);
        perror("Command execution failed");
        exit(EXIT_FAILURE);
    }
}

// Parent closes all pipe descriptors
for (int i = 0; i < 2 * num_pipes; i++) {
    close(pipefds[i]);
}

// Wait for all child processes to exit
for (int i = 0; i < num_cmds; i++) {
    wait(NULL);
}
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nComplex Shell > ");
        if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

        input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline

        if (strcmp(input_buf, "exit") == 0) break;
        if (strlen(input_buf) == 0) continue;

        execute_complex_pipeline(input_buf);
    }

    return 0;
}

```

OUTPUT :

```
Complex Shell > cat combined.log
ls: cannot access '/invalid_folder': No such file or directory
/tmp:
snap-private-tmp
systemd-private-e74ed260026d4f9bad18fb3662be3473-chrony.service-a8gQSW
systemd-private-e74ed260026d4f9bad18fb3662be3473-systemd-logind.service-0vzArS

Complex Shell > grep root < /etc/passwd | cut -d: -f1 > output_users.txt

Complex Shell > cat output_users.txt
root

Complex Shell > ls -la /usr /invalid_dir 2>&1 | grep -i "no such" > error_summary.txt
grep: such: No such file or directory

Complex Shell > cat error_summary.txt

Complex Shell > exit
abhis@MIKHAIL-37:~$ valgrind --leak-check=full ./combined_complex
==922== Memcheck, a memory error detector
==922== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==922== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==922== Command: ./combined_complex
==922==

Complex Shell > exit
==922==
==922== HEAP SUMMARY:
==922==    in use at exit: 0 bytes in 0 blocks
==922==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
==922==
==922== All heap blocks were freed -- no leaks are possible
==922==
==922== For lists of detected and suppressed errors, rerun with: -s
==922== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```